VERIQTA

Cloud Security Lab:

20 Real Misconfigurations to Find and Fix

Hands-On AWS Security Investigations for Junior Cloud, DevOps and Security Engineers



20 REAL FINDINGS	STEP-BY-STEP FIXES	HANDS-ON LABS	PRODUCTION BEST PRACTICES

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

IAM User With Administrator Access

Misconfiguration: An application user has unrestricted AWS permissions through the AdministratorAccess managed policy.

1. What Is IAM?

- AWS Identity and Access Management (IAM) controls who can access AWS resources and what actions they can perform.
- It uses users, groups, roles, and policies to grant permissions.

IAM Access Model



2. Investigate the Issue

- Check what policy is attached to the user and confirm if it has administrator access.

```
$ aws iam list-attached-user-policies \
    --user-name lab-app-user
```

Expected Output

```
{
  "AttachedPolicies": [
   {"PolicyName": "AdministratorAccess"}
  ]
}
```

3. Fix the Misconfiguration

- Remove the administrator policy.
- Attach a least-privilege policy that allows only the required actions (e.g., read/write to a specific S3 bucket).

```
$ aws iam detach-user-policy \
    --user-name lab-app-user \
    --policy-arn arn:aws:iam::aws:policy/
    AdministratorAccess

$ aws iam attach-user-policy \
    --user-name lab-app-user \
    --policy-arn arn:aws:iam::123456789012:
    policy/LabAppS3Access
```

4. Verify the Fix

- Test an allowed action (should succeed).

```
$ aws s3 ls s3://lab-app-data
```

- Test a restricted action (should fail).

```
$ aws ec2 describe-instances
```

Expected: AccessDenied
(You are not authorized to perform this action)

Production Lesson

A compromised application credential should not provide control over the entire AWS account.

VERIQTA

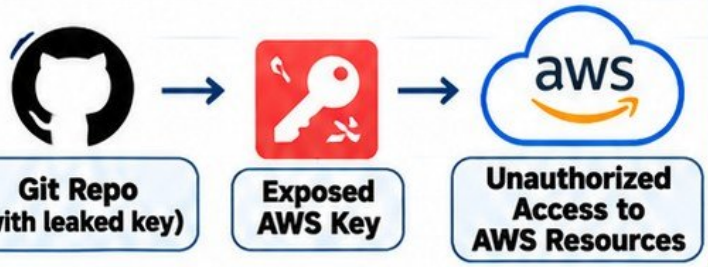
Long-Lived Access Keys Exposed in Git

Misconfiguration: An AWS access key is committed to a Git repository.

1. Why It Matters

- An AWS access key can be used to make AWS API requests.
- If exposed, an attacker can access your AWS resources (e.g., read S3, launch EC2, delete data).
- Long-lived keys are risky because they remain valid until you revoke them.

The Risk in a Simple Flow



2. Investigate the Leaked Key

- Find the secret in your repository history and confirm the key is still active.

```
# Search for possible keys in history
$ git log -p | grep -i "AKIA"
```

```
# Check the key status in IAM
$ aws iam list-access-keys \
    --user-name lab-deployer
```

Example Output

AccessKeyId	Status	CreateDate
AKIA*********	Active	2026-10-01

3. Fix the Misconfiguration

- Immediately stop the exposed key.

```
$ aws iam update-access-key \
    --user-name lab-deployer \
    --access-key-id AKIA*********\
    --status Inactive
```

- Delete the old key after replacement.

```
$ aws iam delete-access-key \
    --user-name lab-deployer \
    --access-key-id AKIA*********\
```

- Remove the secret from Git history (e.g., using git filter-repo or BFG).
- Replace it with a secure method (AWS IAM role, SSM, or Secrets Manager).

4. Verify the Fix

- Confirm the old key no longer works (it should fail).

```
$ AWS_ACCESS_KEY_ID=AKIA********* \
  AWS_SECRET_ACCESS_KEY=xxxx \
  aws sts get-caller-identity
```

Expected: An error such as "The security token included in the request is invalid."

- Verify the application now uses a safer method (e.g., IAM role with temporary credentials).

Production Lesson

Deleting a secret from the latest Git commit does not revoke it.

Root Account Without MFA

Misconfiguration: The AWS account root user has no multifactor authentication (MFA).

1. Why It Matters

- The root user is the most powerful identity in an AWS account.
- It can access all AWS services, change account settings, and bypass permissions.
- Without MFA, a stolen root password can give an attacker full control of your account.

Root vs IAM User

Root User	IAM User
Full account control	Limited permissions (least privilege)
Bypasses IAM limits	Used for daily work (e.g., deployment)
Must have MFA	Can be restricted with policies
Used only for account setup and recovery	

2. Check MFA Status

- Use IAM to view your account MFA status.

```
$ aws iam get-account-summary
```

- Look for the value: "AccountMFAEnabled".

Example Output (Partial)

```
{
    "SummaryMap": {
        "AccountMFAEnabled": false,
        ...
    }
}
```

3. Fix: Enable MFA for Root

1. Sign in to the AWS root account (https://aws.amazon.com/).
2. Go to My Security Credentials → Multi-factor authentication (MFA).
3. Register an MFA device (Authenticator app or hardware key).

4. Verify and Secure Recovery

- Re-run the command to confirm:

```
$ aws iam get-account-summary
```

- Ensure "AccountMFAEnabled": true.
- Also secure recovery options (verified email and phone) and store them safely.

Production Lesson

The root user should not perform routine deployment or administration.

EC2 Instance Role With Excessive Permissions

Misconfiguration: An EC2 application instance has a role permitting s3:* on all resources.

1. How It Works

- EC2 uses an instance profile (role) to give applications temporary AWS credentials (via the instance metadata service).
- The role's IAM policies define what the application can access (e.g., S3, RDS, etc.).

EC2 Role Access Flow



EC2 gets temporary credentials from the IAM role to access AWS resources.

2. Investigate the Issue

- Check the role and attached policies for excessive permissions.

```
$ aws ec2 describe-instances \
  --instance-ids i-0abc123
```

```
$ aws iam list-attached-role-policies \
  --role-name app-ec2-role
```

Problem Policy (Overprivileged)

```
{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*"
}
```

Issue: Full access to ALL S3 buckets and actions.

3. Fix the Misconfiguration

- Replace the broad policy with a least-privilege policy for only the required bucket and actions.

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject",
             "s3:PutObject"],
  "Resource":
  "arn:aws:s3:::app-data-bucket/*"
}
```

4. Verify the Fix

Test allowed access (should succeed):

```
$ aws s3 ls s3://app-data-bucket
```

Test an unauthorized bucket (should fail):

```
$ aws s3 ls s3://other-bucket
```

Expected: AccessDenied
(You are not authorized to access this bucket)

Production Lesson

An application compromise becomes more dangerous when its instance role is overprivileged.

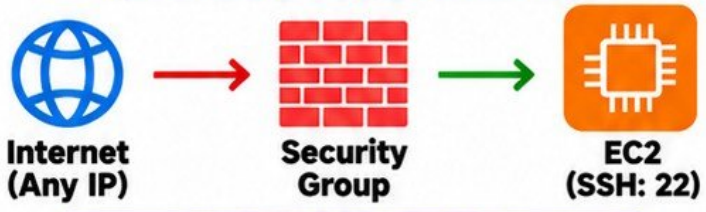
SSH Port 22 Open to the Internet

Misconfiguration: An EC2 security group allows SSH connections from 0.0.0.0/0.

1. What Are Security Groups?

- A virtual firewall in AWS that controls inbound and outbound network traffic to resources (e.g., EC2, RDS).
- You define rules by port, protocol (e.g., TCP), and source (e.g., IP address or another security group).

Security Group in Action



Allows or blocks traffic based on rules (port, protocol, source).

2. Detect the Misconfiguration

- Check the security group rules for an open SSH port (22) from any IP.

```
$ aws ec2 describe-security-groups \
  --group-ids sg-0abc123
```

- Look for an inbound rule like:

```
{
  "IpProtocol": "tcp",
  "FromPort": 22,
  "ToPort": 22,
  "CidrIp": "0.0.0.0/0"
}
```

Exposed Rule vs. Reachable Service

Exposed Rule (Configuration)	Security group allows SSH (port 22) from 0.0.0.0/0.
Reachable (Service)	The SSH service is actually running and accessible from the internet (e.g., confirmed with `nc` or `nmap`).

3. Fix the Misconfiguration

- Remove the broad SSH rule.
- Use AWS Systems Manager (Session Manager) for secure instance access (no open SSH port).

```
$ aws ec2 revoke-security-group-
  ingress \
  --group-id sg-0abc123 \
  --protocol tcp --port 22 \
  --cidr 0.0.0.0/0
```

4. Verify the Fix

- Confirm the SSH rule is removed:

```
$ aws ec2 describe-security-groups \
  --group-ids sg-0abc123
```

- Use Session Manager to access the instance:

```
$ aws ssm start-session --target i-0abc123
```

Access works via Session Manager, and SSH is no longer open to the internet.

Production Lesson

Management ports should not be exposed to the entire internet.

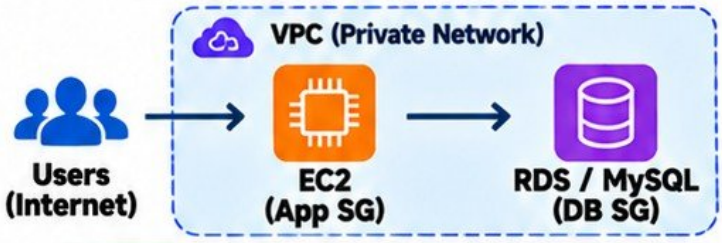
Database Port 3306 Exposed Publicly

Misconfiguration: A database security group permits MySQL connections from any IPv4 address.

1. How It Should Work

- The application and database should communicate privately inside a VPC.
- The database (MySQL port 3306) should only allow traffic from the application's security group (or trusted subnets), not the internet.

Proper VPC Architecture



Only App SG can access DB on port 3306.

2. Find the Misconfiguration

- Check the security group rules for an open MySQL port (3306) from 0.0.0.0/0.

```
$ aws ec2 describe-security-groups \
  --group-ids sg-db123
```

- Look for a rule like:

```
{
  "IpProtocol": "tcp",
  "FromPort": 3306,
  "ToPort": 3306,
  "CidrIp": "0.0.0.0/0"
}
```

3. Inspect Endpoint and Routing

- Check the database endpoint and confirm if it's publicly reachable.

```
$ aws rds describe-db-instances \
  --db-instance-identifier lab-mysql
```

- Review network routing (subnet and route table) to ensure the database is in a private subnet.

```
$ aws ec2 describe-route-tables \
  --filters Name=vpc-id,Values=vpc-abc123
```

4. Fix the Misconfiguration

- Remove the public rule (0.0.0.0/0).
- Allow only the application's security group to access port 3306.

```
$ aws ec2 authorize-security-group-ingress \
  --group-id sg-db123\
  --protocol tcp --port 3306\
  --source-group sg-app123
```

5. Verify the Fix

Confirm the new rule allows only the app security group.

```
$ aws ec2 describe-security-groups \
  --group-ids sg-db123
```

Test from the application (should succeed):

```
$ mysql -h <db-endpoint> -u appuser -p
```

Test from an unauthorized source (should fail):

Connection timed out / Access denied

Production Lesson

Database access should be based on workload identity and network requirements, not broad public access.

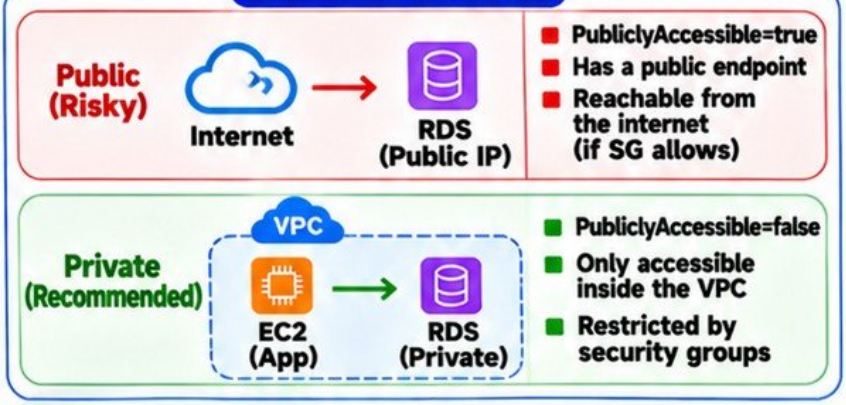
RDS Database Marked Publicly Accessible

Misconfiguration: An Amazon RDS database has PubliclyAccessible=true.

1. Key Concepts

- **PubliclyAccessible:** An RDS setting that assigns a public IP and allows internet routing to the database.
- **Network reachability:** Actual ability to connect, which also depends on security group rules, subnet routes, and NACLs.
- A database can be public but still restricted by security groups (or private but still exposed if misconfigured).

Access Scenarios



2. Inspect RDS Settings

- Check the RDS configuration:

```
$ aws rds describe-db-instances \
  --db-instance-identifier lab-db
```

- Look for key values in the output:

```
{
  "DBInstanceIdentifier": "lab-db",
  "PubliclyAccessible": true,
  "DBSubnetGroup": "lab-subnet-group",
  "Endpoint": "lab-db.xxxx.rds.amazonaws.com"
}
```

3. Investigate Network Controls

- Confirm the full network exposure, not just the RDS setting:
  - ✓ **Security Group:** Is port 3306 (MySQL) open to 0.0.0.0/0 or a wide range?
  - ✓ **Subnet Group:** Are the subnets public (with a route to an Internet Gateway)?
  - ✓ **Routing:** Check the route table for 0.0.0.0/0 → igw-xxxx.
  - ✓ **Endpoint:** Resolve the RDS endpoint and confirm it has a public IP.

4. Fix the Misconfiguration

- Move the database to a private network configuration:
  1. Set PubliclyAccessible to false.
  2. Use private subnets (no route to an Internet Gateway).
  3. Allow access only from the application's security group.
  4. Test the change in a non-production environment or a maintenance window.

5. Verify the Fix

- ✓ Confirm the RDS setting is updated:

```
$ aws rds describe-db-instances --db-instance-
  identifier lab-db --query "DBInstances[0].PubliclyAccessible"
```

- ✓ Ensure the value is **false**.
- ✓ Verify application connectivity from the EC2/app.
- ✗ Test from an external network (should fail):

```
Error: Connection timed out / Access denied
```

Production Lesson

A private database requires multiple correctly configured network controls.

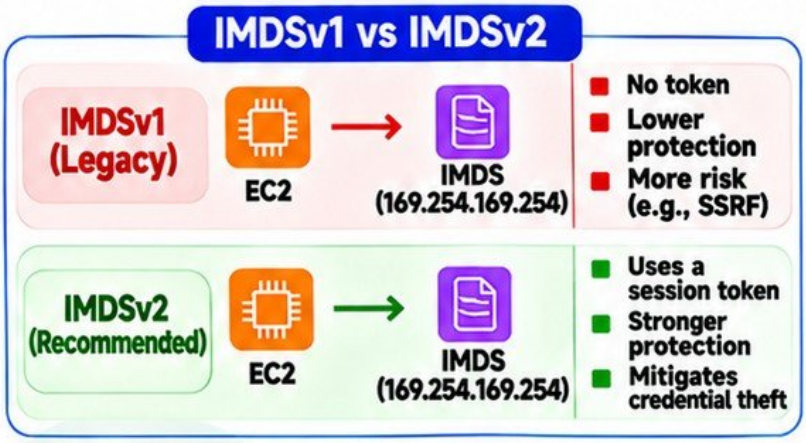
EC2 Instance Metadata Service Allows IMDSv1

Misconfiguration: An EC2 instance permits legacy Instance Metadata Service Version 1 (IMDSv1) requests.

1. How It Works

- EC2 provides instance metadata (IMDS) at 169.254.169.254 for applications running on the instance.
- Applications use this service to obtain temporary AWS credentials (from the instance role), region, and other instance information.
- IMDSv2 adds a session token and request protection to reduce the risk of unauthorized access (e.g., SSRF).

IMDSv1 vs IMDSv2



2. Inspect HttpTokens Setting

- Check the instance metadata option using AWS CLI:

```
$ aws ec2 describe-instances \
  --instance-ids i-0abc123 \
  --query "Reservations[].Instances[].
  MetadataOptions"
```

- Sample output (misconfigured):

```
{
  "HttpTokens": "optional",
  "HttpEndpoint": "enabled",
  "HttpPutResponseHopLimit": 1
}
```

3. Identify the Issue



- Look for instances where:

`HttpTokens = optional`

- This means the instance still allows IMDSv1 requests, which is a security risk, especially if the application can be tricked into making metadata requests (e.g., via SSRF).

4. Fix the Misconfiguration

1. Validate that the application and workloads support IMDSv2.
2. Require IMDSv2 (block IMDSv1):

```
$ aws ec2 modify-instance-metadata-
  options --instance-id i-0abc123 \
  --http-tokens required
```

3. (Optional) Set a lower hop limit:

```
$ ... --http-put-response-hop-limit 1
```

5. Verify the Fix

- Confirm the setting is updated:

```
$ aws ec2 describe-instances \
  --instance-ids i-0abc123 \
  --query "Reservations[].Instances[].
  MetadataOptions.HttpTokens"
```

Output: "required"

- Test application functionality to ensure it can still retrieve credentials and operate normally.

Production Lesson

Metadata protection reduces credential exposure risk in certain application compromise scenarios.

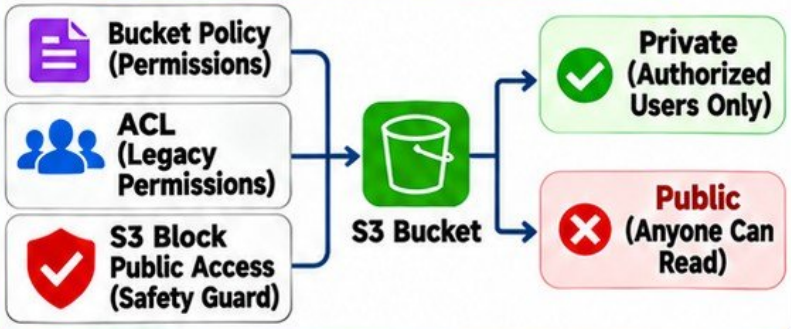
S3 Bucket Allows Public Read Access

Misconfiguration: An S3 bucket policy grants public object-reading permissions.

1. Key Concepts

- **Bucket Policy:** A resource-based policy that defines who can access a bucket and what actions are allowed.
- **ACL (Access Control List):** A legacy permission model that can also grant public access.
- **S3 Block Public Access:** Account or bucket-level controls that prevent public access, even if a policy or ACL tries to allow it.

How S3 Access Is Controlled



2. Inspect the Configuration

- Check the bucket policy:

```
$ aws s3api get-bucket-policy \
  --bucket lab-public-bucket
```

- Look for a public principal and s3:GetObject, e.g.:

```
{
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::lab-public-bucket/*"
}
```

- Check public access settings:

```
$ aws s3api get-public-access-block \
  --bucket lab-public-bucket
```

3. Identify the Issue

Issue: The bucket allows public read access because the policy grants s3:GetObject to all principals ("*").

4. Fix the Misconfiguration

1. Remove or update the bucket policy to restrict access to specific IAM principals.
2. Enable S3 Block Public Access at the bucket level:

```
$ aws s3api put-public-access-block \
  --bucket lab-public-bucket \
  --public-access-block-configuration \
  "BlockPublicAcls=true,IgnorePublicAcls=true,\
   BlockPublicPolicy=true,RestrictPublicBuckets=true"
```

3. Review and clean up any public ACLs (if present).

5. Verify the Fix

- Confirm the bucket policy no longer allows public access:

```
$ aws s3api get-bucket-policy --bucket lab-public-bucket
```

- Confirm Block Public Access is enabled:

```
$ aws s3api get-public-access-block --bucket lab-public-bucket
```

- Test an unauthorized read (should fail):

```
$ curl https://lab-public-bucket.s3.amazonaws.com/test.txt
```

Expected: AccessDenied

Production Lesson

A single exposed bucket can disclose backups, customer records, or internal documents.

S3 Bucket Without Default Encryption

Misconfiguration: An S3 bucket lacks an explicit default encryption configuration required by company policy.

1. Key Concepts

- **Encryption at Rest:** Protects data stored in S3 so it can only be read with the correct keys.
- **SSE-S3:** AWS-managed encryption (default for new uploads). Simpler, no key management.
- **SSE-KMS:** Uses AWS Key Management Service (KMS) with customer-managed keys (CMK). Provides more control, access policies, and auditing.

S3 Encryption Options



2. Inspect the Configuration

- Check if the bucket has an explicit default encryption rule:

```
$ aws s3api get-bucket-encryption \
  --bucket lab-data-bucket
```

- If no rule exists, the output returns:

```
An error occurred (ServerSideEncryption
ConfigurationNotFoundException) when
calling the GetBucketEncryption operation:
The server-side encryption configuration
was not found.
```

3. Important AWS Behavior

Amazon S3 automatically encrypts new objects with SSE-S3, even if you do not set an explicit bucket encryption rule.

What This Means

- Your data is still encrypted at rest by default (SSE-S3).
- The finding is about policy compliance (e.g., requiring SSE-KMS), not a lack of encryption.

Common Misunderstanding

- No explicit encryption rule does NOT mean the data is unencrypted (flag this only if your policy requires an explicit configuration, such as SSE-KMS).

4. Fix the Misconfiguration

1. Create or use a KMS key (CMK):

```
$ aws kms create-key \
  --description "S3 encryption key for lab"
```

2. Configure default encryption with SSE-KMS:

```
$ aws s3api put-bucket-encryption \
  --bucket lab-data-bucket \
  --server-side-encryption-configuration '{
    "Rules": [{
      "ApplyServerSideEncryptionByDefault": {
        "SSEAlgorithm": "aws:kms",
        "KMSMasterKeyID": "arn:aws:kms:region:acct\
        :key/1234abcd-5678-90ef-..."
      }
    }]
}'
```

5. Verify the Fix

- Confirm the default encryption is now set:

```
$ aws s3api get-bucket-encryption \
  --bucket lab-data-bucket
```

Expected (key part):

```
"SSEAlgorithm": "aws:kms",
"KMSMasterKeyID": "arn:aws:kms:..."
```

- Upload a test file and check its encryption:

```
$ aws s3api head-object \
  --bucket lab-data-bucket --key test.txt
```

Expected:

```
"ServerSideEncryption": "aws:kms"
```

Production Lesson — Security findings must reflect current AWS behavior, not outdated assumptions.

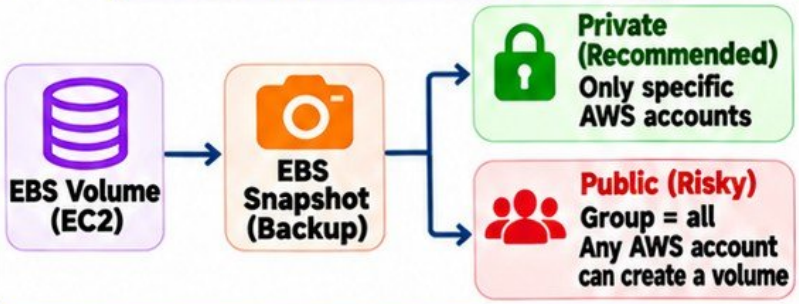
EBS Snapshot Shared With Every AWS Account

Misconfiguration: An EBS snapshot has public create-volume permissions (shared with all AWS accounts).

1. Key Concepts

- **EBS Snapshot:** A point-in-time backup of an EBS volume.
- **What It Can Contain:** Operating system files, application data, configuration files, logs, and potential credentials (e.g., keys, secrets, or tokens).
- **Risk:** If a snapshot is shared publicly, any AWS account can create a new EBS volume from it and read the data.

How Snapshot Sharing Works



2. Inspect Snapshot Permissions

- Check the snapshot's create-volume permissions:

```
$ aws ec2 describe-snapshot-attribute \
  --snapshot-id snap-0abc123def456 \
  --attribute createVolumePermission
```

- Example output (misconfigured):

```
{
  "CreateVolumePermissions": [
    {
      "Group": "all"
    }
  ]
}
```

3. Identify the Issue

Issue: The create-volume permissions include Group = all, which makes the snapshot publicly accessible to every AWS account.

Why This Is Dangerous

- Anyone can create a volume from the snapshot.
- The volume can be attached to an EC2 instance and the data read.
- Can expose OS files, application data, and secrets (e.g., database credentials, API keys, or config files).

4. Fix the Misconfiguration

1. Remove public sharing (Group = all):

```
$ aws ec2 modify-snapshot-attribute \
  --snapshot-id snap-0abc123def456 \
  --attribute createVolumePermission
  --operation-type remove
  --group-names all
```

2. (Optional) Share only with approved AWS account(s):

```
$ aws ec2 modify-snapshot-attribute \
  --snapshot-id snap-0abc123def456 \
  --attribute createVolumePermission
  --operation-type add
  --user-ids 123456789012
```

5. Verify the Fix

- Confirm the snapshot is no longer public:

```
$ aws ec2 describe-snapshot-attribute \
  --snapshot-id snap-0abc123def456
  --attribute createVolumePermission
```

Expected output:

```
{
  "CreateVolumePermissions": []
}
```

- Verify only authorized accounts (if any) are listed, not Group = all.

Production Lesson

Backups and snapshots need access controls as strong as production data.

Secrets Stored in EC2 User Data

Misconfiguration: An EC2 launch script contains a hardcoded database password.

1. Key Concepts

- EC2 User Data is supplied to instances at launch (via metadata service) and runs during first boot.
- It's meant for initialization (e.g., installing software, configuring apps), not for storing secrets.
- User data is accessible from within the instance (IMDS) and can be viewed by users with access to instance configuration or logs.

How EC2 User Data Works



Not a secure place for secrets because it can be read from the instance, metadata service, or logs.

2. Inspect the User Data Script

- Check for embedded credentials (e.g., database passwords):

```
#!/bin/bash
export DB_PASSWORD="P@ssw0rd123"
# install and configure application
```

- Example output (simulated):

```
$ grep -i password userdatash, sh
export DB_PASSWORD="P@ssw0rd123"
```

3. Investigate Who Can Access

- Check who can access instance configuration and related logs (e.g., EC2 console, CloudTrail, SSM, logs).
- Look for IAM dsets /roles with excessive permissions.



Key Sources:

- EC2 console (User Data)
- CloudTrail logs
- Systems Manager (SSM)
- Instance logs / metadata

4. Fix the Misconfiguration

1. Rotate the exposed credential immediately.
2. Retrieve secrets from AWS Secrets Manager (using an appropriately scoped instance role).

```
# ssecretsmanager getsecret-value \
  --secret-id db-credentials \
  --region us-east-1
```

3. Update the launch configuration to remove hardcoded secrets.

5. Verify the Fix

- Confirm the application works with the new secret (no errors, correct connectivity).
- Check the instance user data and launch configuration for no plaintext secrets.
- Validate access controls (IAM, CloudTrail, logs) are in plae.

Expected result:

No hardcoded secrets. Secrets retrieved from AWS Secrets Manager. Application running successfully.

Production Lesson

Moving a password into an environment variable does not automatically make it secure.

CloudTrail Logging Disabled

Misconfiguration: A required CloudTrail trail is not logging.

1. Key Concepts

- AWS CloudTrail records account activity and API events (e.g., who did what, when, from where, and which resource).
- It helps with security monitoring, incident investigation, compliance, and audit requirements.
- A trail delivers logs to Amazon S3 (and optionally CloudWatch Logs and EventBridge) for long-term retention and analysis.

How CloudTrail Logging Works



- ✓ Records management, data, and configuration API calls.
- ✓ Stores logs in S3 for long-term audit and investigation.
- ✓ Supports security, compliance, and incident response.

2. Inspect the Trail Status

- Check the logging status of the trail:

```
$ aws cloudtrail get-trail-status \
  --name org-trail
```

- Example output (misconfigured):

```
{
  "IsLogging": false,
  "LatestDeliveryTime": null
}
```

3. Identify the Issue

Issue: IsLogging=false means the CloudTrail trail exists but is not currently recording events.

Common Causes

- ✓ Trail logging was manually stopped.
- ✓ Trail was misconfigured or deleted and recreated without logging enabled.
- ✓ Insufficient permissions to deliver logs to S3.
- ✓ Trail created but never started.

4. Fix the Misconfiguration

1. Start or restore trail logging:

```
$ aws cloudtrail start-logging \
  --name org-trail
```

2. Verify the delivery destination (S3 bucket):

```
$ aws cloudtrail get-trail \
  --name org-trail \
  --query "S3BucketName"
```

3. Ensure the S3 bucket policy allows CloudTrail to write logs (if needed).

5. Verify the Fix

- ✓ Confirm logging is enabled:

```
$ aws cloudtrail get-trail-status \
  --name org-trail
```

Expected output (key part):

```
{
  "IsLogging": true,
  "LatestDeliveryTime": "2026-11-05T10:24:03Z"
}
```

- ✓ Generate a harmless test API event (e.g., describe EC2 instances):

```
$ aws ec2 describe-instances --max-results 1
```

- ✓ Check the S3 log bucket and confirm a new CloudTrail log file appears (e.g., within a few minutes).

Production Lesson — Event history and long-term trail logging serve different purposes. Critical audit coverage must be verified.

CloudTrail Logs Stored in an Unprotected Bucket

Misconfiguration: A logging bucket permits overly broad deletion of CloudTrail log objects.

1. Key Concepts

- CloudTrail logs are audit records of AWS API activity (who, what, when, and where).
- If logs can be deleted or modified, attacks can be hidden, investigations can fail, and compliance evidence can be lost.
- Logging buckets must enforce least privilege, retention, and tamper protection.

How the Logging Flow Works



- CloudTrail delivers log files to an S3 bucket.
- The bucket must prevent unauthorized deletion.
- Retention and Object Lock keep logs intact and verifiable.

2. Inspect the Bucket Configuration

- Check the bucket policy and permissions:

```
$ aws s3api get-bucket-policy \
  --bucket org-cloudtrail-logs
```

- Check object lock and retention:

```
$ aws s3api get-object-lock-configuration \
  --bucket org-cloudtrail-logs
```

- Review the bucket ACL (if used):

```
$ aws s3api get-bucket-acl --bucket \
  org-cloudtrail-logs
```

3. Identify the Issue

Issue: One or more identities have unnecessary permissions to delete CloudTrail log objects.

What to Look For

- Policy statements with "s3:DeleteObject" or "s3:DeleteObjectVersion".
- Overly broad principals (e.g., "Principal": "*").
- Missing or disabled S3 Object Lock / retention controls.

4. Fix the Misconfiguration

1. Remove or restrict delete permissions in the bucket policy (use least privilege):

```
{
   "Effect": "Allow",
   "Action": ["s3:PutObject"],
   "Principal": {"Service": "cloudtrail.amazonaws.com"},
   "Resource": "arn:aws:s3:::org-cloudtrail-logs/*"
}
```

2. Enable S3 Object Lock (if required by policy):

```
$ aws s3api put-object-lock-configuration \
   --bucket org-cloudtrail-logs
   --object-lock-configuration
   '{"ObjectLockEnabled":"Enabled"}'
```

3. Configure a retention rule (example):

```
$ aws s3api put-bucket-lifecycle-configuration \
   --bucket org-cloudtrail-logs
   --lifecycle-configuration file://retention.json
```

5. Verify the Fix

- Confirm CloudTrail can still deliver logs (authorized access):

```
$ aws cloudtrail get-trail-status \
  --name org-trail
```

- Test unauthorized deletion (should fail):

```
$ aws s3 rm s3://org-cloudtrail-logs/test.log
```

Expected output:

An error occurred (AccessDenied) when calling the DeleteObject operation: Access Denied

- Verify new log files are being delivered to the bucket (within a few minutes):

```
$ aws s3 ls s3://org-cloudtrail-logs/AWSLogs/
```

Production Lesson — Audit logs must remain trustworthy even during a security incident.

GuardDuty Disabled in an Active AWS Region

Misconfiguration: Amazon GuardDuty is not enabled in a region hosting workloads.

1. Key Concepts

- Amazon GuardDuty is a threat detection service that continuously analyzes AWS, network, and account activity to find suspicious behavior.
- It uses machine learning, threat intelligence, and behavior analysis to detect potential attacks, compromise, and misuse.
- GuardDuty is regional, so it must be enabled in every AWS region where you run workloads.

How GuardDuty Works



- ✓ Analyzes logs and events (e.g., CloudTrail, VPC Flow Logs, DNS logs, EKS audit logs).
- ✓ Detects suspicious or malicious activity.
- ✓ Generates findings with severity and recommended actions.

2. Inspect the Detector Configuration

- List existing GuardDuty detectors in the current region:

```
$ aws guardduty list-detectors
  --region us-east-1
```

- Example output (detector exists):

```
{
  "DetectorIds": ["12abc34d567e890f1234"]
}
```

- If the region has no detector, the output shows an empty list:

```
{
  "DetectorIds": []
}
```

3. Identify the Issue

Issue: No GuardDuty detector is enabled in an active workload region (e.g., eu-west-1).

How to Confirm

- ✓ Review the list of active AWS regions and workloads (e.g., EC2, RDS, EKS, VPC).
- ✓ Run list-detectors in each region.
- ✓ Identify regions with no detector (missing coverage).
- ✓ Check organization settings (if using AWS Organizations) for centralized GuardDuty.

4. Fix the Misconfiguration

1. Enable GuardDuty in the affected region:

```
$ aws guardduty create-detector \
  --enable \
  --region eu-west-1
```

2. (Optional) Enable and manage at the organization level:

```
$ aws guardduty create-organization-admin-
  account --admin-account-id 123456789012
```

3. Ensure auto-enable for new regions (if required):

```
$ aws guardduty update-organization-
  configuration \
  --auto-enable true
```

5. Verify the Fix

- ✓ Confirm the detector is enabled:

```
$ aws guardduty list-detectors \
  --region eu-west-1
```

Expected output:

```
{
  "DetectorIds": ["a1b2c3d4e5f6g7h8i9j0"]
}
```

- ✓ Generate a supported sample finding (e.g., using the AWS GuardDuty sample finding):

```
$ aws guardduty create-sample-findings \
  --detector-id a1b2c3d4e5f6g7h8i9j0 \
  --finding-types Recon:EC2/PortProbeUnprotectedPort
```

- ✓ Verify the finding in the AWS Console (GuardDuty → Findings) or via CLI:

```
$ aws guardduty list-findings \
  --detector-id a1b2c3d4e5f6g7h8i9j0
```

Production Lesson — Security monitoring coverage must account for every active AWS region.

Production RDS Database Without Deletion Protection

Misconfiguration: A production database has deletion protection disabled.

1. Key Concepts

- Amazon RDS is a managed database service for production workloads.
- Deletion protection prevents a database instance from being accidentally or intentionally deleted.
- Backups (automated or manual) provide recovery options if data is lost or corrupted.

How the Risk Happens



- Without deletion protection, an RDS instance can be deleted with a single command or console action.
- If backups are missing or untested, recovery can be slow, incomplete, or impossible.

2. Inspect the RDS Configuration

- Check the deletion protection setting:

```
$ aws rds describe-db-instances \
  --db-instance-identifier prod-db
```

- Example output (key fields):

```
{
  "DBInstanceIdentifier": "prod-db",
  "DeletionProtection": false,
  "BackupRetentionPeriod": 7
}
```

- Also check backup and recovery settings:

```
$ aws rds describe-db-instances \
  --db-instance-identifier prod-db \
  --query "DBInstances[0].[AutomatedBackups
  RetentionPeriod,LatestRestorableTime]"
```

3. Identify the Issue

Issue: DeletionProtection=false on a production RDS database.

Also Review

- BackupRetentionPeriod: Is it aligned with company policy (e.g., 7, 14, or 35 days)?
- LatestRestorableTime: Confirms you can recover to a recent point in time.
- Snapshot strategy: Ensure regular manual snapshots for critical environments.

4. Fix the Misconfiguration

1. Enable deletion protection:

```
$ aws rds modify-db-instance \
  --db-instance-identifier prod-db
  --deletion-protection
```

2. Review and adjust backup settings (if needed):

```
$ aws rds modify-db-instance \
  --db-instance-identifier prod-db
  --backup-retention-period 14
```

3. (Optional) Create a manual snapshot before changes:

```
$ aws rds create-db-snapshot \
  --db-snapshot-identifier prod-db-prechange
  --db-instance-identifier prod-db
```

5. Verify the Fix

- Confirm deletion protection is enabled:

```
$ aws rds describe-db-instances \
  --db-instance-identifier prod-db
  --query "DBInstances[0].DeletionProtection"
```

Expected output: `true`

- Verify backup and recovery readiness:

```
$ aws rds describe-db-instances \
  --db-instance-identifier prod-db
  --query "DBInstances[0].{BackupRetention:
   BackupRetentionPeriod,LatestRestore:
   LatestRestorableTime]"
```

- (Non-destructive) Test recovery readiness (e.g., create a snapshot and confirm it exists):

```
$ aws rds describe-db-snapshots \
  --db-instance-identifier prod-db
```

Production Lesson

Deletion protection is not a replacement for tested backups.

ECR Repository With Mutable Image Tags

Misconfiguration: A production container repository permits overwriting existing image tags.

1. Key Concepts

- Amazon ECR (Elastic Container Registry) stores and manages docker container images.
- An image tag (e.g., v1.0, latest) is a human-friendly label for a specific image.
- Tags can be mutable (can be overwritten) or immutable (cannot be changed once pushed).
- In production, mutable tags can lead to untested or malicious images being deployed.

How Image Tags Work



- A tag points to a specific container image (an image digest).
- Mutable tags can be overwritten with a different image.
- Immutable tags keep a tag fixed to the original image.

2. Inspect the Repository Settings

- Check the ECR repository configuration:

```
$ aws ecr describe-repositories \
    --repository-names prod-app
```

- Example output (key part):

```
{
  "repositories": [
    {
      "repositoryName": "prod-app",
      "imageTagMutability": "MUTABLE"
    }
  ]
}
```

3. Identify the Issue

Issue: imageTagMutability=MUTABLE on a production repository.

Why It's a Problem

- An existing tag (e.g., v1.0) can be overwritten with a different image.
- Deployments may pull an untested or malicious image with the same tag.
- It breaks traceability and can hide unauthorized changes.

4. Fix the Misconfiguration

1. Enable tag immutability for the production repository:

```
$ aws ecr put-image-tag-mutability \
    --repository-name prod-app \
    --image-tag-mutability IMMUTABLE
```

2. (Optional) Apply to multiple repositories:

```
$ aws ecr put-image-tag-mutability \
    --repository-name prod-api \
    --image-tag-mutability IMMUTABLE
```

3. Confirm the setting:

```
$ aws ecr describe-repositories \
    --repository-names prod-app \
    --query "repositories[0].imageTagMutability"
```

5. Verify the Fix

- Confirm the repository is now immutable:

```
$ aws ecr describe-repositories \
    --repository-names prod-app \
    --query "repositories[0].imageTagMutability"
```

Expected output:

```
"IMMUTABLE"
```

- Test that overwriting an existing tag is rejected (using a disposable lab repository):

```
$ docker push <account>.dkr.ecr.<region>.amazonaws.com/
  lab-app:v1.0
```

Expected result (example):

denied: cannot overwrite existing image tag because the repository is immutable

Production Lesson

Deployments should reference verified image digests when artifact identity matters.

Lambda Function With Administrator Permissions

Misconfiguration: A Lambda execution role has administrator-level AWS permissions.

1. Key Concepts

- AWS Lambda runs your code without managing servers.
- A Lambda function uses an execution role (IAM role) to access AWS services (e.g., S3, DynamoDB, CloudWatch).
- The role should follow the principle of least privilege, with only the minimum actions required.

How Lambda Permissions Work



- The execution role defines what AWS actions the function can perform.
- Overly broad permissions (e.g., AdministratorAccess) give the function unnecessary and high-risk access.

2. Inspect the Function Role

- Get the Lambda function configuration:

```
$ aws lambda get-function-configuration \
  --function-name prod-handler
```

- Check the role and its policies:

```
$ aws iam get-role \
  --role-name lambda-prod-role
```

```
$ aws iam list-attached-role-policies
  --role-name lambda-prod-role
```

3. Identify the Issue

Issue: The Lambda execution role is attached to administrator-level permissions (e.g., AdministratorAccess).

What to Look For

- Attached policy list includes "AdministratorAccess" or "PowerUserAccess".
- Inline policy with "Action": "*" and "Resource": "*".
- Permissions that are broader than what the function actually needs (e.g., access to unrelated services).

4. Fix the Misconfiguration

1. Create a least-privilege IAM policy (for example: read/write to a specific S3 bucket):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Effect": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::prod-app-data/*"
  }]
}
```

2. Update the Lambda function to use a new role with the least-privilege policy:

```
$ aws lambda update-function-configuration \
  --function-name prod-handler \
  --role arn:aws:iam::<account-id>:role/lambda-s3-role
```

3. (Optional) Detach the AdministratorAccess policy from the old role:

```
$ aws iam detach-role-policy\
  --role-name lambda-prod-role
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess
```

5. Verify the Fix

- Confirm the function now uses the new role:

```
$ aws lambda get-function-configuration \
  --function-name prod-handler
  --query 'Role' --output text
```

Expected output (example):
`arn:aws:iam::<account-id>:role/lambda-s3-role`

- Run an authorized test event (e.g., access the allowed S3 object) and confirm it succeeds.

```
$ aws lambda invoke \
  --function-name prod-handler
  --payload '{}' response.json
```

Expected result:
StatusCode: 200 (successful execution)

- Try an unauthorized operation (e.g., list IAM users) and confirm it is denied:

```
$ aws iam list-users
```

Expected result:
An error occurred (AccessDenied)

Production Lesson — A vulnerable serverless function should not become an AWS account takeover path.

KMS Key Policy Grants Excessive Administrative Access

Misconfiguration: An AWS KMS key policy delegates sensitive key administration to an overly broad IAM principal.

1. Key Concepts

- AWS KMS (Key Management Service) creates, manages, and controls encryption keys.
- A key policy is the primary access control for a KMS key and defines who can administer and use the key.
- IAM policies can also grant KMS permissions, but access is only effective when allowed by the key policy.

How KMS Access Works



- The key policy sets the boundary for key access (primary control).
- IAM policies can grant permissions, but cannot override a restrictive key policy.

2. Inspect the Key Policy

- Retrieve the key policy:

```
$ aws kms get-key-policy \
  --key-id <key-id> \
  --policy-name default
```

- Example output (risky statement):

```
{
  "Sid": "AllowAdminAccess",
  "Effect": "Allow",
  "Principal": { "AWS": "*" },
  "Action": "kms:*",
  "Resource": "*"
}
```

3. Identify the Issue

Issue: The key policy grants overly broad administrative access (e.g., Principal="*" or a wide IAM role) with actions like kms:*.

Look For These Signs

- Principal set to "*" (all principals) or an unnecessarily broad IAM role/group.
- Administrative actions (e.g., kms:CreateKey, kms:ScheduleKeyDeletion, kms:PutKeyPolicy, kms:DisableKey, kms:EnableKeyRotation).
- No separation between key administration and key usage.

4. Fix the Misconfiguration

1. Update the key policy to follow least privilege (separate admin and usage):

```
{
  "Statement": [
    {
      "Sid": "KeyAdminOnly",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:
      role/kms-admin-role" },
      "Action": ["kms:DescribeKey","kms:EnableKeyRotation",
      "kms:PutKeyPolicy","kms:CreateGrant"],
      "Resource": "*"
    },
    {
      "Sid": "KeyUsageOnly",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:
      role/app-role" },
      "Action": ["kms:Encrypt","kms:Decrypt",
      "kms:GenerateDataKey*","kms:ReEncrypt*"],
      "Resource": "*"
    }
  ]
}
```

2. Apply the updated policy:

```
$ aws kms put-key-policy \
  --key-id <key-id> \
  --policy-name default \
  --policy file://kms-policy.json
```

5. Verify the Fix

- Confirm the key policy is updated:

```
$ aws kms get-key-policy \
  --key-id <key-id> \
  --policy-name default
```

Expected check:

- Admin access is limited to the approved role (e.g., kms-admin-role).
- Usage permissions are scoped to the required actions only.

- Test authorized operations (should succeed):

```
$ aws kms encrypt \
  --key-id <key-id> \
  --plaintext fileb://test.txt \
  --output text --query CiphertextBlob
```

- Test an unauthorized admin action (should fail):

```
$ aws kms schedule-key-deletion --key-id <key-id>
```

Expected result:
An error occurred (AccessDeniedException)

Production Lesson

Encryption is only as trustworthy as the access controls protecting its keys.

Final Security Investigation: Multiple Misconfigurations in One AWS Environment

Scenario: A fictional payment application has been deployed with multiple security weaknesses.



1. Investigation Goals

- Discover at least four planted misconfigurations using AWS CLI evidence.
- Explain the security impact of each finding.
- Prioritize remediation based on exposure and potential impact.
- Apply fixes in an isolated, authorized lab environment.
- Verify that each security control works without breaking the application.
- Produce a short security findings report containing evidence, remediation, and verification results.

2. Key AWS CLI Checks (Evidence)

`$ aws ec2 describe-security-groups \ --group-ids sg-xxxx`	Check SSH (port 22) exposure.
`$ aws iam list-attached-role-policies \ --role-name ec2-app-role`	Identify excessive IAM permissions.
`$ aws s3api get-bucket-policy \ --bucket payment-app-data`	Review S3 bucket access policy.
`$ aws rds describe-db-instances \ --db-instance-identifier prod-db`	Check RDS public accessibility.

3. Findings and Remediation Plan

#	Finding (Misconfiguration)	Security Impact	Priority	Remediation (Fix)
1	EC2 with exposed SSH (0.0.0.0/0) and excessive IAM	Unauthorized access and potential AWS account takeover.	High	Restrict SSH to a trusted IP/VPC and apply least-privilege IAM role.
2	S3 bucket with public read policy	Data exposure, loss of confidential payment files.	High	Remove public access, enable block public access, and apply a least-privilege bucket policy.
3	RDS publicly accessible (PubliclyAccessible=true)	Direct internet access to the database, risk of data breach.	High	Set PubliclyAccessible=false, place in a private subnet, and use a security group.
4	EC2 role with broad permissions (e.g., AdministratorAccess)	A compromised instance can control or delete AWS resources.	Medium	Replace with a least-privilege policy for only required services.

4. Verification Steps

- Confirm SSH (port 22) is no longer publicly open.
- Validate the S3 bucket is not publicly accessible.
- Ensure the RDS instance is private and reachable only from the application (EC2).
- Confirm the EC2 role has only required permissions.
- Run the application end-to-end to verify availability.

5. Security Findings Report (Template)

- Executive Summary: What was found and the risk level.
- Findings: Each misconfiguration with CLI evidence (command output).
- Remediation: What was changed and why.
- Verification: How you confirmed the fix works.
- Conclusion: Current security posture and next steps.

Final Challenge

Can you secure the environment while preserving application availability?